

SYSTEM ARCHITECTURE

How the layers are arranged and why each decision was made

1.0 SYSTEM OVERVIEW

The **Smart Corporate Equipment & Asset Reservation System** manages high-value asset checkout processes with strict governance and automated rule enforcement. It is a lightweight, responsive web application combining a clean REST API backend with a modern HTML5 / Vanilla CSS frontend styled as a physical inventory ledger.

2.0 SYSTEM COMPONENTS

▸ Frontend UI — Presentation Layer

Vanilla HTML5, CSS3, and JavaScript. Renders real-time asset availability, handles user input, performs client-side validation, and asynchronously fetches data from the API.

▸ API Server — Application Layer

Node.js + Express. Handles core business logic, input sanitisation, XSS / injection checks, rate-limiting, and RESTful routing.

▸ Data Store — Data Layer

Encapsulated `database.js` backed by flat CSV files (`assets.csv` / `reservations.csv`) via `csv-parse` and `csv-stringify` . Chosen so reviewers can inspect live data in Excel without database provisioning.

3.0 DATA FLOW

- 01 User Action** — user selects an asset and submits a reservation date range via the UI.
- 02 Client Validation** — frontend checks dates are valid (start $\geq$ today, end $>$ start).
- 03 API Request** — POST with `assetId` , `userId` , `startDate` , `endDate` .
- 04 Server Validation** — past dates, XSS safety, asset maintenance status, duration cap, user quota, overlap check.
- 05 Data Mutation** — record created with status `APPROVED` or `PENDING_APPROVAL` .
- 06 Response** — 201 Created with the new reservation object.
- 07 UI Update** — success toast displayed; asset list reloaded.

4.0 TECHNOLOGY CHOICES

TECHNOLOGY	CHOICE	RATIONALE
Backend	Node.js + Express	Fast scaffolding; non-blocking I/O ideal for API servers.
Frontend	Vanilla JS / HTML / CSS	Zero build step; runs directly in any browser without compilation.
Testing	Jest + Supertest	Robust assertion library; Supertest allows endpoint testing without a live server port.

TECHNOLOGY	CHOICE	RATIONALE
Architecture	Monolithic MVC-like	Separated routing, logic, and data layers — right balance of structure for this scope.

5.0 TRADE-OFFS & KNOWN LIMITATIONS

CSV-BACKED PERSISTENCE

Uses flat CSV files instead of a relational database. Allows reviewers to watch data update in Excel in real-time. Data integrity is protected by an **asynchronous write queue** and **atomic rename operations** (`fs.renameSync`) — writing to a `.tmp` first and flipping it over the real file, so a crash cannot produce a corrupted CSV.

TEST ISOLATION

When `NODE_ENV=test` , the database targets `test-assets.csv` / `test-reservations.csv` so running Jest never touches live demo data.

AUTHENTICATION

Roles and the Login Screen are purely frontend constructs for the demo. **Anyone can type any name and check Admin without cryptographic verification.** The backend strictly validates and sanitises all payloads regardless of role.